แผนการเรียนรายวิชาวิทยาการคำนวณ/คอมพิวเตอร์

ระดับชั้น: ประถมศึกษาตอนปลาย (ป.4-6)
ระยะเวลา: 4 คาบเรียน (คาบละ 50 นาที)
วิชา: วิทยาการคำนวณ/คอมพิวเตอร์
หน่วยการเรียนรู้ : Computational Thinking

มาตรฐานการเรียนรู้และตัวชี้วัด

มาตรฐาน ว 4.2 เข้าใจและใช้แนวคิดเชิงคำนวณในการแก้ปัญหาที่พบในชีวิตจริงอย่างเป็นขั้นตอนและเป็นระบบ

มาตรฐาน ว 4.1 ใช้เทคโนโลยีสารสนเทศและการสื่อสารในการเรียนรู้และการทำงานได้อย่างมีประสิทธิภาพ

จุดประสงค์การเรียนรู้

ด้านความรู้ (Knowledge: K)

1. อธิบายองค์ประกอบของ Computational Thinking ได้
2. เข้าใจหลักการของ Algorithm และ Flowchart
3. เข้าใจความหมายของ Logic / Condition

ด้านทักษะ (Process: P)

1. แยกปัญหาเป็นส่วนย่อยได้อย่างเป็นระบบ
2. เขียนขั้นตอนการทำงานแบบลำดับ (Step-by-step) ได้
3. สร้าง Flowchart จากสถานการณ์ที่กำหนดได้
4. Debug (หาข้อผิดพลาด) ของกระบวนการคิดได้

ด้านคุณลักษณะ (Attitude: A)

1. มีความคิดเป็นระบบ
2. กล้าลองผิดลองถูก
3. ทำงานร่วมกับผู้อื่นได้
4. มีความรับผิดชอบต่องานที่ได้รับ

คาบเรียนที่ 31 – องค์ประกอบของแนวคิดเชิงคำนวณ (Computational Thinking) และ การแบ่งปัญหาใหญ่เป็นปัญหาย่อย (Decomposition)

นำเข้าสู่บทเรียน (10 นาที)

ครูถามนำ เช่น

ถ้าต้อง “จัดกระเป๋าไปทัศนศึกษา” เราควรเริ่มจากอะไร

ถ้ามีงานใหญ่ 1 งาน เราจะทำให้จัดการง่ายขึ้นได้อย่างไร

แนวคิดเชิงคำนวณ (Computational Thinking) คืออะไร?

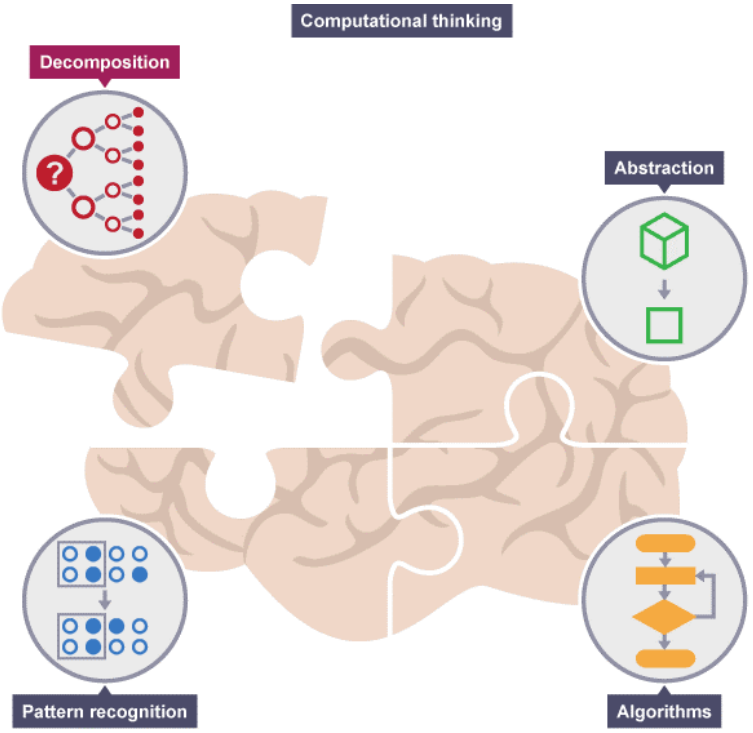
คือ "กระบวนการคิด" เพื่อแก้ปัญหาอย่างเป็นระบบ โดยแบ่งปัญหาที่ซับซ้อนให้กลายเป็นส่วนย่อยๆ แล้วหาวิธีการแก้ไขที่ชัดเจน จนแม้แต่ "หุ่นยนต์" หรือ "คอมพิวเตอร์" (ที่ไม่มีความรู้สึกเหมือนมนุษย์) ก็สามารถทำตามคำสั่งนั้นจนสำเร็จได้

แนวคิดเชิงคำนวณมีองค์ประกอบที่สำคัญ 4 ส่วน ได้แก่

1. การแบ่งปัญหาใหญ่เป็นปัญหาย่อย (Decomposition)
2. การพิจารณารูปแบบ (Pattern Recognition)
3. การคิดเชิงนามธรรม (Abstraction)
4. การออกแบบอัลกอริทึม (Algorithm)

นำเข้าสู่บทเรียนการแบ่งปัญหาใหญ่เป็นปัญหาย่อย

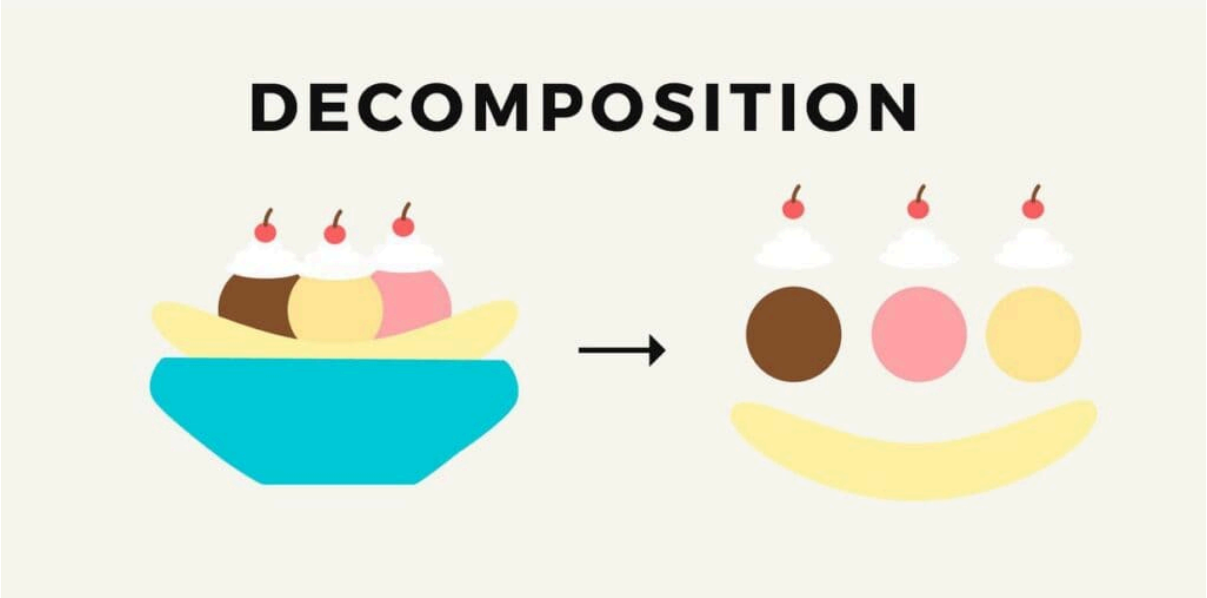
ใครเคยเจองานใหญ่ๆ แล้วรู้สึก 'ท้อ' หรือ 'ทำไม่ถูก' บ้าง? เช่น จะต้องจัดห้องนอนที่รกมากๆ หรือต้องเขียนรายงานส่งครูทั้งเล่ม... ยิ่งมองงานใหญ่เท่าไหร่ เราก็ยิ่งอยากถอนหายใจใช่ไหม?



1. **การแบ่งปัญหาใหญ่เป็นปัญหาย่อย (Decomposition) (30 นาที)**

คือการแตกปัญหาหรือระบบที่ซับซ้อนออกเป็นส่วนย่อยที่เล็กกว่า เพื่อให้ง่ายต่อการทำความเข้าใจ การวิเคราะห์ และการแก้ไขปัญหาทีละส่วนอย่างเป็นขั้นตอน ช่วยให้จัดการปัญหาได้รวดเร็ว ลดความซับซ้อน และเพิ่มประสิทธิภาพในการทำงานหรือเขียนโปรแกรม

เปรียบเสมือนกุญแจไขปัญหาที่ช่วยปลดล็อกความซับซ้อน แบ่งปัญหาใหญ่ที่ยากจะเข้าใจออกเป็นปัญหาย่อยๆ ที่เข้าใจง่ายและจัดการได้สะดวก เปรียบเสมือนการแยกชิ้นส่วนของเครื่องจักรออกเป็นส่วนประกอบต่างๆ เพื่อศึกษาการทำงานของแต่ละส่วน



1.1 หลักการของ Decomposition

- **แบ่งปัญหาออกเป็นส่วนย่อยๆ** หลักการสำคัญของ Decomposition คือการแบ่งปัญหาที่ใหญ่และซับซ้อนออกเป็นปัญหาย่อยๆ ที่เล็กลงและจัดการได้ง่ายกว่า โดยแต่ละปัญหาย่อยจะเป็นองค์ประกอบหรือส่วนประกอบของปัญหาใหญ่นั้น เช่น การแบ่งโจทย์ปัญหาออกเป็นข้อย่อยๆ การแยกส่วนของซอฟต์แวร์เป็นโมดูลย่อยๆ
- **จัดการปัญหาย่อยตามลำดับก่อนหลัง** หลังจากแบ่งปัญหาใหญ่ออกเป็นปัญหาย่อยแล้ว ต้องวิเคราะห์และจัดลำดับการแก้ไขปัญหาย่อยเหล่านั้นให้เป็นขั้นตอน เพื่อแก้ไขไปทีละส่วนจนครบถ้วน บางครั้งอาจต้องแก้ไขปัญหาบางส่วนก่อน เพื่อนำผลลัพธ์ไปใช้ในการแก้ไขปัญหาส่วนอื่นๆ ต่อไป
- **มองเห็นภาพรวมขององค์ประกอบย่อย** การ Decomposition ยังช่วยให้สามารถมองเห็นภาพรวมขององค์ประกอบย่อยของปัญหาได้ชัดเจนขึ้น ทำให้เข้าใจความสัมพันธ์และการทำงานร่วมกันขององค์ประกอบเหล่านั้น รวมถึงช่วยให้สามารถระบุจุดเชื่อมโยงหรือส่วนที่เกี่ยวข้องกันได้ดีขึ้น

- **แยกแยะส่วนสำคัญจากส่วนรอง** การแตกปัญหาออกเป็นส่วนย่อย ช่วยให้สามารถแยกแยะส่วนที่เป็นแก่นสำคัญของปัญหาออกจากส่วนที่เป็นรายละเอียดรอง นำไปสู่การให้ความสำคัญกับส่วนที่สำคัญเป็นอันดับแรก แล้วจึงค่อยพิจารณารายละเอียดรองในภายหลัง
- **ทำให้ปัญหาซับซ้อนกลายเป็นเรื่องง่าย** แก่นของ Decomposition คือการทำให้ปัญหาที่ยากและซับซ้อนกลายเป็นชุดของปัญหาย่อยๆ ที่ง่ายต่อการทำความเข้าใจและจัดการมากขึ้น ด้วยการแบ่งออกเป็นส่วนย่อย จึงช่วยลดภาระและความยากลงได้

ฉะนั้น การแตกปัญหาตามหลักการของ Decomposition จึงเป็นทักษะสำคัญในการคิดแก้ปัญหาและการคิดเชิงคำนวณ ช่วยให้สามารถจัดการกับความซับซ้อนของปัญหาต่างๆ ได้อย่างเป็นระบบและมีประสิทธิภาพมากขึ้น

1.2 5 ประโยชน์ของ Decomposition

1. **ลดความซับซ้อน (เหมือนการกินพิซซ่า):**
   คำอธิบาย: "เวลาเราได้พิซซ่าถาดใหญ่มาก ใครจะเอาเข้าปากทั้งถาดได้ล่ะ? เราต้องหั่นเป็นชิ้นๆ ก่อนใช่ไหมครับ? งานใหญ่ก็เหมือนพิซซ่า ถ้าหั่นเป็นชิ้นเล็กๆ เราก็กินได้ง่ายและไม่เลอะเทอะ"
2. **จัดการง่าย (เหมือนการต่อเลโก้):**
   คำอธิบาย: "ถ้าเราจะต่อเลโก้เป็นปราสาท 1,000 ชิ้น แล้วเทกองรวมกัน เราคงหาชิ้นส่วนไม่เจอ แต่ถ้าเราแยกสี แยกส่วนฐาน ส่วนกำแพง ส่วนหลังคาไว้ เราจะต่อเสร็จเร็วมาก เพราะเราหยิบมาใช้ได้ทันที"
3. **เพิ่มประสิทธิภาพ (เหมือนการแบ่งงานในกลุ่ม):**
   คำอธิบาย: "ถ้าเราทำงานคนเดียวมันช้า แต่ถ้าเราย่อยงานแล้วแบ่งให้เพื่อนช่วยกันทำ เช่น คนหนึ่งคีบ คนหนึ่งวาง คนหนึ่งจดบันทึก งานของเราจะเสร็จไวขึ้น 3 เท่าเลยครับ!"
4. **ส่งเสริมการคิดอย่างมีระบบ (เหมือนการหาของในห้อง):**
   คำอธิบาย: "คนเก่งจะรู้ว่าต้องเริ่มที่จุดไหนก่อน-หลัง ถ้าเราไม่ย่อยงาน เราจะวิ่งไปวิ่งมาทำนั่นทำนี่จนงง แต่ถ้าเราย่อยงาน เราจะรู้ว่า: 'อ๋อ! ต้องทำข้อ 1 ก่อน แล้วค่อยไปข้อ 2' แบบนี้ไม่มีหลงทางแน่นอน"
5. **พัฒนาทักษะการแก้ปัญหา (เหมือนการเลเวลอัปในเกม):**
   คำอธิบาย: "เวลาเราเล่นเกม เราจะผ่านด่านไปทีละด่านใช่ไหมครับ? การย่อยงานก็เหมือนการผ่านด่านย่อยๆ ถ้าเราผ่านด่านเล็กๆ ได้บ่อยๆ ความมั่นใจเราจะเพิ่มขึ้น จนในที่สุดเราก็พิชิตงานใหญ่ได้สบายๆ เลย!"

เทคนิคเพิ่มเติมสำหรับคุณครู:
หากเด็กๆ ยังไม่เห็นภาพ ให้ลองชวนเขา "ย่อยการแปรงฟัน" ดูครับ เช่น

1. บีบยาสีฟัน
2. ขัดฟันหน้า
3. ขัดฟันใน
4. บ้วนปาก

ถ้าทำสลับขั้นตอนจะเป็นยังไง? (บ้วนปากก่อนบีบยาสีฟัน) เด็กจะเข้าใจทันทีว่า "ลำดับและชิ้นงานย่อยนั้นสำคัญแค่ไหน"

ใบงานที่ 1 : การแยกย่อยปัญหา (Decomposition) (10 นาที | 10 คะแนน)

ชื่อ: _____เลขที่_____ชั้น_____

จุดประสงค์
ให้นักเรียนสามารถแยกปัญหาใหญ่เป็นงานย่อยได้
คำชี้แจง ให้นักเรียนเลือก 1 สถานการณ์ แล้วแยกออกเป็นงานย่อยให้ชัดเจน

สถานการณ์ตัวอย่าง เลือก 1 ข้อ

- การเตรียมตัวไปทัศนศึกษา
- การจัดโต๊ะอ่านหนังสือ
- การจัดกิจกรรมวันเกิดในห้องเรียน
- การปลูกต้นไม้ 1 กระถาง

แบบบันทึก ปัญหาใหญ่: _____(สถานการณ์ที่เลือก)

งานย่อยที่ต้องทำ	ใครรับผิดชอบ	อุปกรณ์ที่ใช้
1		
2		
3		
4		
5		

คำถามท้ายใบงาน

1. ถ้าไม่แยกงานย่อย จะเกิดปัญหาอะไร

.....
.....

2. งานย่อยใดควรทำก่อน เพราะอะไร

.....
.....

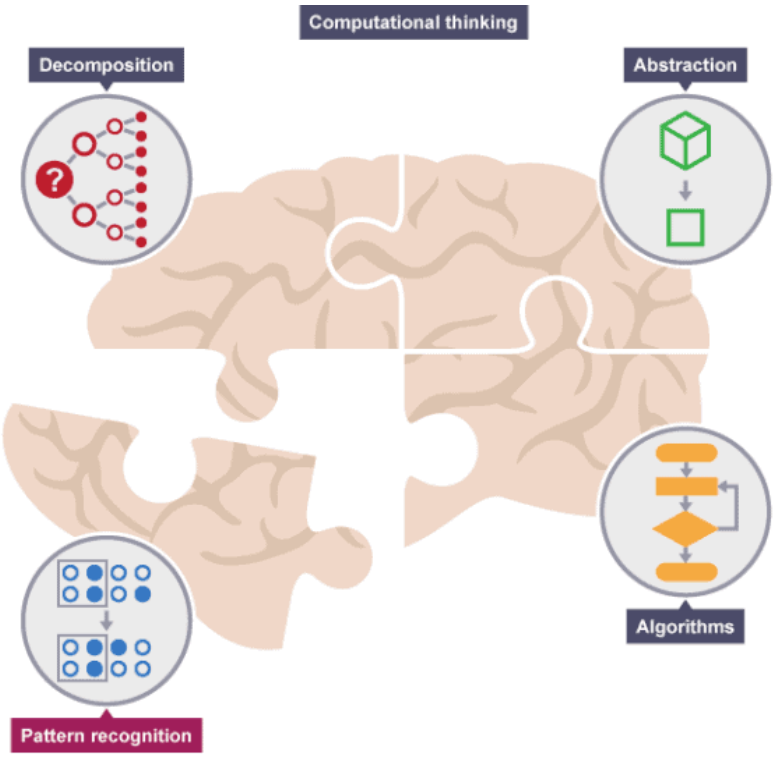
เฉลยแนวทาง *ไม่มีคำตอบตายตัว แต่ควรเห็นการแบ่งงานออกเป็นส่วนย่อยที่ชัดเจนและจัดลำดับได้*

คะแนนรวม: ..... / 10 คะแนน ลายเซ็นครู: .....

คาบเรียนที่ 32 – การพิจารณารูปแบบ (Pattern Recognition) และ การคิดเชิงนามธรรม (Abstraction Thinking)

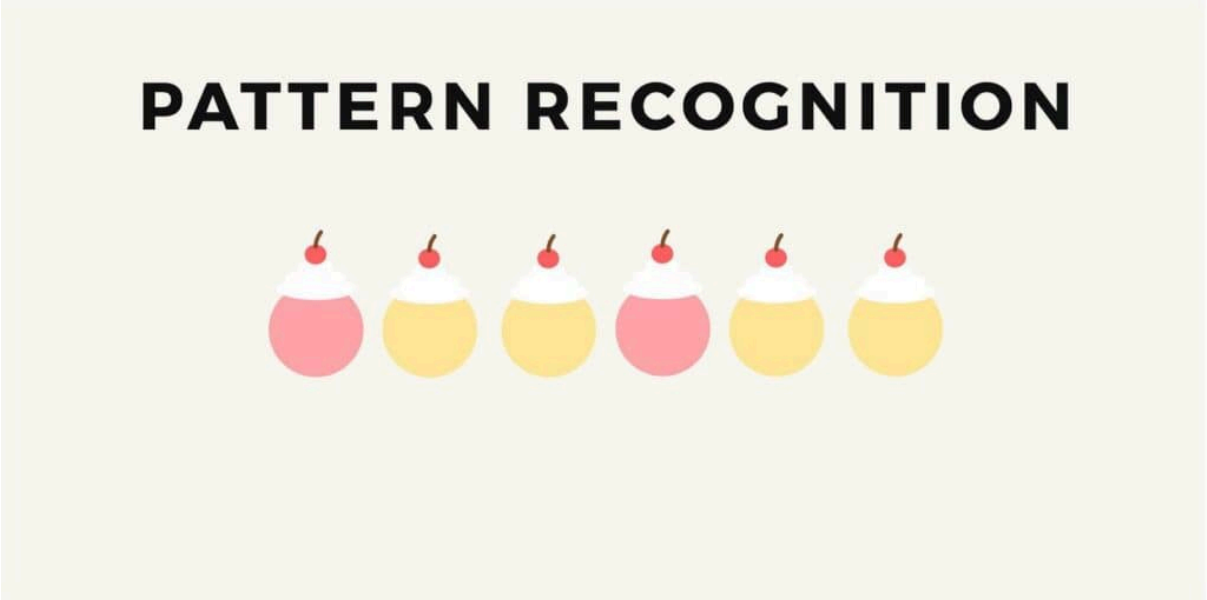
นำเข้าสู่บทเรียน (10 นาที)
"นักเรียนครับ ถ้าครูบอกว่า ครูมีชุดตัวเลข *(2, 4, 6, 8, ...)* นักเรียนบอกได้ไหมครับว่าตัวต่อไปคืออะไร? ...แล้วถ้าครูบอกว่า ครูมีชุดท่าทาง คีบ-ยก-หมุน-ปล่อย แล้วครูทำซ้ำแบบนี้ 3 รอบ หุ่นยนต์ของเราต้องทำอะไรบ้าง? นี่แหละครับคือหัวใจของการที่เราจะสั่งหุ่นยนต์ได้เก่งๆ เราต้อง 'มองเห็นรูปแบบ' (Pattern) ให้เจอ!"

เนื้อหาที่จะสอนในขั้นนำ:
ความหมาย: Pattern Recognition คือการมองหาสิ่งที่ "ซ้ำเดิม" หรือ "ความสัมพันธ์" ในปัญหา
ทำไมต้องเรียน: เพราะหุ่นยนต์ทำงานตาม "รูปแบบ" ที่เราวางไว้ ถ้าเรามองเห็นรูปแบบ เราก็แค่สั่งงานมันแค่ครั้งเดียว แต่ให้มันทำซ้ำไปเรื่อยๆ ได้



2. การจับจำแนกรูปแบบ (Pattern Recognition) (30 นาที)

เป็นทักษะสำคัญสำหรับนักเขียนโปรแกรม (Programmer) ทักษะนี้ช่วยให้มองเห็นรูปแบบ ลำดับ ความสัมพันธ์ และโครงสร้างในข้อมูล โค้ด และอัลกอริทึม ทักษะนี้เปรียบเสมือนเข็มทิศนำทาง ช่วยให้เขียนโปรแกรมที่มีประสิทธิภาพ เข้าใจง่าย บำรุงรักษาง่าย และแก้ไขได้ง่าย



2.1 ประเภทของ Pattern Recognition

สามารถแบ่งออกได้เป็น 4 ประเภทหลัก ดังนี้

2.1.1. Spatial patterns: รูปแบบเชิงพื้นที่

รูปแบบนี้เกี่ยวข้องกับรูปร่าง ลวดลาย สีสัน และโครงสร้างเชิงพื้นที่ ตัวอย่าง:

- การจดจำใบหน้า
- การอ่านตัวอักษร
- การวิเคราะห์ภาพถ่าย
- การออกแบบกราฟิก
- การเล่นเกม

2.1.2. Temporal patterns: รูปแบบเชิงเวลา

รูปแบบนี้เกี่ยวข้องกับลำดับ เหตุการณ์ วัฏจักร และการเปลี่ยนแปลงตามเวลาตัวอย่าง:

- การวิเคราะห์ตลาดหุ้น
- การคาดการณ์สภาพอากาศ
- การวิเคราะห์เพลง
- การเขียนโปรแกรม
- การติดตามการขนส่ง

2.1.3. Sequential patterns: รูปแบบเชิงลำดับ

รูปแบบนี้เกี่ยวข้องกับขั้นตอน สูตร กฎเกณฑ์ และลำดับขั้นตอน ตัวอย่าง:

- การเขียนโค้ด
- การประกอบเฟอร์นิเจอร์

- การทำอาหาร
- การเล่นดนตรี
- การทำความสะอาดบ้าน

2.1.4. Hierarchical patterns: รูปแบบเชิงลำดับชั้น

รูปแบบนี้เกี่ยวข้องกับโครงสร้าง องค์ประกอบ และระดับ ตัวอย่าง:

- การจัดระเบียบข้อมูล
- การเขียนโครงเรื่อง
- การออกแบบเว็บไซต์
- การจัดการโครงการ
- การวิเคราะห์ระบบ

2.2 ประโยชน์ของ Pattern Recognition

- **ช่วยให้แก้ปัญหาได้ "เร็วขึ้น" (Efficiency):**

  คำอธิบาย: "ถ้าเราเจอโจทย์ที่คล้ายกับโจทย์เดิมที่เราเคยแก้ได้แล้ว เราก็ไม่ต้องเสียเวลาคิดใหม่ตั้งแต่ศูนย์ เราแค่ใช้ 'วิธีเดิม' ที่เคยสำเร็จมาจัดการได้เลย เหมือนนักกีฬาที่ซ้อมท่าเดิมจนชำนาญ พอถึงเวลาแข่งก็ทำได้อัตโนมัติ"

- **ช่วยให้เรา "ทำนายอนาคต" ได้ (Prediction):**

  คำอธิบาย: "เมื่อเราเห็นรูปแบบที่ชัดเจน เราจะเริ่มเดาได้ว่า 'ต่อไปจะเกิดอะไรขึ้น' เช่น ถ้าหุ่นยนต์คีบของสีแดงแล้วไปวางที่ตะกร้าซ้ายทุกครั้ง เราก็เดาได้เลยว่าถ้าครั้งนี้หุ่นยนต์เจอสีแดง มันต้องไปวางตะกร้าซ้ายแน่นอน!"

- **ช่วยให้ "เขียนคำสั่งน้อยลง" (Simplicity):**

  คำอธิบาย: "แทนที่จะเขียนคำสั่ง 100 บรรทัดซ้ำๆ เราสามารถรวมคำสั่งที่เป็นรูปแบบเดียวกันไว้เป็น 'กลุ่มก้อน' แล้วสั่งให้หุ่นยนต์ทำซ้ำ (Loop) ได้ ทำให้เราทำงานน้อยลง แต่ได้งานเท่าเดิม"

- **ช่วยให้ "ลดข้อผิดพลาด" (Accuracy):**

  คำอธิบาย: "เมื่อเรามองเห็นรูปแบบ เราจะรู้ว่าอะไรคือสิ่งปกติ อะไรคือสิ่งผิดปกติ เช่น ถ้าหุ่นยนต์เดินไปทางขวา 3 ก้าวเสมอ แต่จู่ๆ วันนี้มันเดินไปทางซ้าย เราจะรู้ทันทีว่า 'เฮ้ย! หุ่นยนต์ต้องมีปัญหาอะไรแน่ๆ' ทำให้เราแก้ปัญหาได้ก่อนที่จะเกิดเรื่องใหญ่"

- **ช่วยให้ "จัดระเบียบความคิด" (Organization):**

  คำอธิบาย: "โลกนี้เต็มไปด้วยข้อมูลมหาศาล Pattern Recognition ช่วยให้เราหยิบข้อมูลที่ 'เหมือนๆ กัน' มาจัดกลุ่มไว้ด้วยกัน ทำให้สมองเราไม่รกและคิดงานได้เป็นระบบมากขึ้นครับ"

ใบงานที่ 2 : การจับจำแนกรูปแบบ (Pattern Recognition) (10 นาที | 10 คะแนน)

ชื่อ: _____เลขที่_____ชั้น_____

คำชี้แจง: ให้นักเรียนสวมบทบาทเป็น "นักสืบ" เพื่อออกตามหารหัสลับที่ซ่อนอยู่ในสถานการณ์ต่างๆ โดยใช้ทักษะ

1. จงเติมคำตอบในช่องว่างที่หายไป ตามรูปแบบ (Pattern):
   - A, B, A, B, A, ... (คำตอบ: __B__)
   - 10, 20, 30, 40, ... (คำตอบ: _50___)
   - คีบ, หมุน, ปล่อย, คีบ, หมุน, ... (คำตอบ: __คีบ__)

2. ในการสั่งหุ่นยนต์คีบของ 5 ชิ้น หากหุ่นยนต์ต้องทำท่าทางเหมือนเดิมทุกครั้ง นักเรียนคิดว่าเราต้องเขียนคำสั่งกี่ครั้ง?
   - [ ] เขียนคำสั่งยาวๆ 5 ครั้ง
   - [ ] เขียนคำสั่งชุดเดียว แล้วสั่งให้ทำซ้ำ (Repeat) 5 ครั้ง

3. การมองเห็น Pattern ช่วยนักเรียนอย่างไร? (เลือกคำตอบที่ถูก)
   - [ ] ทำให้งานเสร็จเร็วขึ้น
   - [ ] ทำให้เราเขียนคำสั่งผิดน้อยลง
   - [ ] ถูกทุกข้อ

ส่วนที่ 2 : ไขรหัสลับพิกัดหุ่นยนต์ (Pattern Matching)

1. ให้นักเรียนพิจารณาลำดับการเคลื่อนที่ของแขนกล xArm ต่อไปนี้ แล้วเติมสิ่งที่ขาดหายไป

   [ยกแขน] -> [คีบ] -> [ยกแขน] -> [คีบ] -> [.....] -> [คีบ]

   คำตอบ: [ยกแขน]

2. หากหุ่นยนต์ถูกตั้งโปรแกรมให้ "หมุนขวา 30 องศา" ทุกๆ 1 นาที ตอนนี้ผ่านไป 3 นาที หุ่นยนต์หมุนไปแล้ว 90 องศา นักเรียนคิดว่าเมื่อครบ "4 นาที" หุ่นยนต์จะอยู่ที่พิกัดกี่องศา?

   คำตอบ: 120 องศา (เพราะรูปแบบคือเพิ่มทีละ 30)

3. ถ้านักเรียนต้องสั่งหุ่นยนต์ให้ "สวัสดี" 100 ครั้ง นักเรียนจะเลือกวิธีใดระหว่าง "เขียนคำสั่งใหม่ 100 บรรทัด" หรือ "หาความซ้ำแล้วสั่งทำซ้ำ (Loop)" เพราะอะไร?

   แนวการตอบ: สั่งทำซ้ำ เพราะเร็วกว่าและไม่เหนื่อย (ประหยัดเวลา/Efficiency)

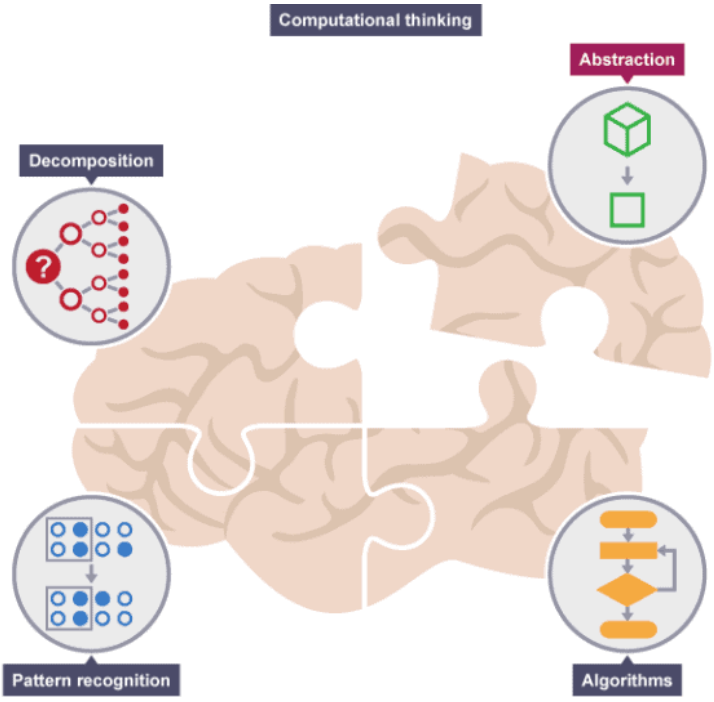
คะแนนรวม: ..... / 10 คะแนน ลายเซ็นครู: .....

การคิดเชิงนามธรรม Abstraction Thinking

นำเข้าสู่บทเรียน (10 นาที)

"นักเรียนครับ ถ้าครูจะสั่งให้หุ่นยนต์ วิ่งจากจุด A ไปจุด B นักเรียนคิดว่าเราต้องบอกหุ่นยนต์ไหมครับว่า 'บนโต๊ะมีแก้วน้ำกี่ใบ' หรือ 'หน้าต่างสีอะไร'?"

"คำตอบคือ... ไม่ต้อง! เพราะสิ่งเหล่านั้นไม่มีผลต่อการเคลื่อนที่ของหุ่นยนต์เลย การที่เรา 'เลือกดูเฉพาะสิ่งที่สำคัญ' (ระยะทาง, ทิศทาง) และ 'ตัดสิ่งที่ไร้สาระออกไป' นี่แหละครับ คือสิ่งที่เรียกว่า Abstraction"



3. การคิดเชิงนามธรรม Abstraction Thinking (30 นาที)

หมายถึง ความสามารถในการมองภาพรวม แยกแยะส่วนประกอบสำคัญ และละเลยรายละเอียดที่ไม่จำเป็น

การแยกแยะและเลือกเอาเฉพาะ "หัวใจสำคัญ" ของปัญหาหรือวัตถุ โดยตัดรายละเอียดที่ไม่จำเป็น (Noise) ออกไป เพื่อให้เราสามารถมองเห็นภาพรวมและแก้ปัญหาได้ง่ายขึ้นโดยไม่หลงทางไปกับรายละเอียดเล็กน้อยที่ไม่ได้ช่วยให้งานสำเร็จ ทักษะนี้ช่วยให้เข้าใจปัญหา วิเคราะห์ข้อมูล และออกแบบโปรแกรมที่มีประสิทธิภาพ



3.1 ประโยชน์ของ Abstraction

1. **แก้ปัญหาได้ตรงจุด**: เพราะเราโฟกัสแค่สิ่งที่สำคัญ งานจึงเสร็จเร็วขึ้น
2. **ลดความสับสน:** ตัดข้อมูลขยะทิ้งไป ทำให้สมองไม่ล้า
3. **ใช้ซ้ำได้ง่าย:** เมื่อเราได้ "สูตรสำเร็จ" ที่เป็นนามธรรม (ตัดรายละเอียดออกหมด) เราจะสามารถนำสูตรนั้นไปใช้กับปัญหาที่คล้ายกันได้เสมอ

3.2 การพัฒนาทักษะ Abstraction Thinking

1. **การฝึกฝนการสังเกต:** ฝึกฝนการสังเกตรายละเอียด รูปแบบ ลำดับ และความสัมพันธ์ในข้อมูล สิ่งของ เหตุการณ์ และธรรมชาติ เช่น สังเกตรูปทรงของใบไม้ ดอกไม้ ผลไม้ สังเกตลำดับขั้นตอนในการทำอาหาร สังเกตความสัมพันธ์ระหว่างสัตว์และสิ่งแวดล้อม
2. **การวิเคราะห์ข้อมูล:** ฝึกฝนการวิเคราะห์ข้อมูล หาความสัมพันธ์ เปรียบเทียบ ค้นหาความคล้ายคลึง เช่น วิเคราะห์ข้อมูลสถิติ เปรียบเทียบสินค้าสองชนิด ค้นหาความคล้ายคลึงของภาษา วัฒนธรรม
3. **การคิดวิเคราะห์:** ฝึกฝนการคิดวิเคราะห์ ตั้งคำถาม หาคำตอบ คาดการณ์ เช่น ตั้งคำถามเกี่ยวกับเหตุการณ์ที่เกิดขึ้น หาคำตอบจากข้อมูลที่มีอยู่ คาดการณ์ผลลัพธ์ของการทดลอง
4. **การฝึกฝนการใช้เครื่องมือ:** ฝึกฝนการใช้เครื่องมือช่วยวิเคราะห์ข้อมูล เช่น โปรแกรมคอมพิวเตอร์ เช่น ใช้โปรแกรม Excel ในการวิเคราะห์ข้อมูล ใช้โปรแกรม Photoshop ในการวิเคราะห์ภาพ
5. **การเล่นเกม:** การเล่นเกมบางประเภท เช่น เกมปริศนา เกมจับคู่ เกมเรียงลำดับ ช่วยฝึกฝนทักษะ Pattern Recognition เช่น เล่นเกมจับคู่ภาพ เล่นเกมเรียงลำดับตัวเลข เล่นเกม Sudoku

School Bot

ใบงานที่ 3 : การคิดเชิงนามธรรม Abstraction Thinking (10 นาที | 10 คะแนน)

ชื่อ: _____**เลขที่**_____**ชั้น**_____

คำชี้แจง: ให้นักเรียนสวมบทบาทเป็น Game Designer เพื่อสร้างระบบเกม โดยการเลือกข้อมูลที่ "จำเป็นต่อการเล่น " และตัดข้อมูลที่ "ไม่เกี่ยวข้อง" ออกไป

ภารกิจที่ 1: สร้างเกม "Robo-Soccer" (หุ่นยนต์เตะบอล)

หากนักเรียนต้องเขียนโปรแกรมสั่งหุ่นยนต์เตะบอล ข้อมูลใดบ้างที่ จำเป็น (สำคัญ) และข้อมูลใดที่ ไม่จำเป็น (ตัดทิ้ง) (ทำเครื่องหมายลงในช่อง)

ข้อมูลที่พบในสถานการณ์	จำเป็นต่อการเตะบอล	ไม่จำเป็น (ตัดทิ้ง)
1. ตำแหน่งของลูกฟุตบอลบนสนาม	[ ]	[ ]
2. ยี่ห้อรองเท้าของผู้ชมข้างสนาม	[ ]	[ ]
3. ระยะห่างระหว่างหุ่นยนต์กับประตู	[ ]	[ ]
4. สีของท้องฟ้าในวันนั้น	[ ]	[ ]
5. พลังงานแบตเตอรี่ที่เหลืออยู่ของหุ่นยนต์	[ ]	[ ]
6. ชื่อตัวการ์ตูนที่ติดอยู่บนตัวหุ่นยนต์	[ ]	[ ]

ภารกิจที่ 2: วาดภาพไอคอน (Icon Abstraction)

ในเกมจะมี "ไอคอน" แทนคำพูด ให้นักเรียนลองวาดรูปเพื่อแทนคำสั่งต่อไปนี้ โดยใช้เส้นให้ น้อยที่สุด แต่ดูแล้วรู้ทันทีว่าคืออะไร

คำสั่งในเกม	วาดรูป Abstraction (ใช้เส้นน้อยๆ)
1. ปรับระดับเสียง (Volume)	[ ]
2. ตั้งค่า (Settings)	[ ]
3. ร้านค้า (Shop)	[ ]

คะแนนรวม: ..... / 10 คะแนน ลายเซ็นครู: .....

คาบเรียนที่ 33-34 – การออกแบบอัลกอริทึม (Algorithm) และการออกแบบ Flowchart

นำเข้าสู่บทเรียน (10 นาที)

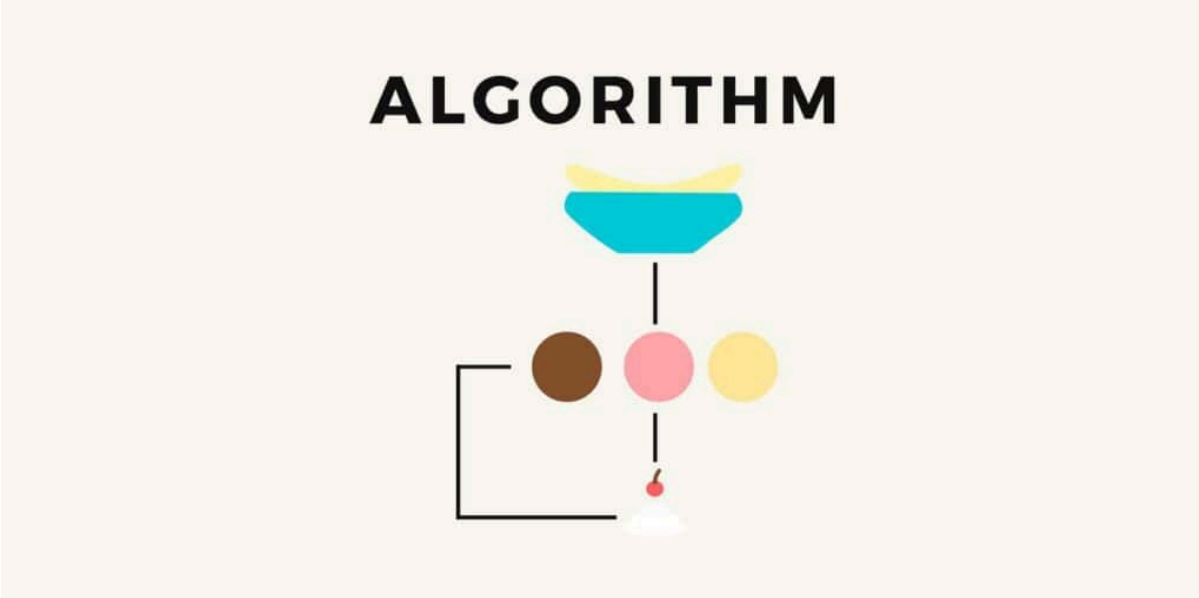
"นักเรียนครับ ถ้าครูสั่งหุ่นยนต์ว่า 'จงทำไข่เจียวให้ครูกินหน่อย' แล้วหุ่นยนต์หยิบไข่ทั้งฟองโยนลงกระทะที่ยังไม่ได้เปิดไฟ นักเรียนว่าเราจะได้กินไข่เจียวไหมครับ?"

"นั่นเป็นเพราะหุ่นยนต์ไม่รู้ 'ลำดับ' ที่ถูกต้องครับ วันนี้เราจะมาเรียนรู้วิธีการเขียน 'คู่มือการทำงาน' ที่ชัดเจนจนใครอ่านก็ทำตามได้สำเร็จ สิ่งนี้นี้เรียกว่า Algorithm ครับ"

4. การออกแบบอัลกอริทึม (Algorithm) (30 นาที)

Algorithm คือ ชุดของคำสั่งหรือขั้นตอนอย่างเป็นลำดับที่ใช้ในการแก้ปัญหาหรือดำเนินการบางอย่าง คำสั่งเหล่านี้จะต้องเป็นไปอย่างชัดเจน ไม่คลุมเครือ และสามารถปฏิบัติได้จริง อีกทั้งยังต้องมีจุดเริ่มต้นและจุดสิ้นสุดที่แน่นอน

Algorithm เป็นคำศัพท์ที่มีความสำคัญอย่างยิ่งในโลกของคอมพิวเตอร์และการเขียนโปรแกรม แต่หลายคนอาจยังไม่เข้าใจความหมายและบทบาทของมันอย่างถ่องแท้ ตัวอย่างของ Algorithm ที่พบเห็นได้ในชีวิตประจำวัน เช่น วิธีการทำอาหาร ขั้นตอนการเดินทางจากจุดหนึ่งไปยังอีกจุดหนึ่ง หรือแม้แต่การสวมเสื้อผ้า ซึ่งแต่ละอย่างล้วนมีขั้นตอนที่เป็นลำดับชัดเจน เมื่อปฏิบัติตามจะทำให้บรรลุผลสำเร็จตามที่ต้องการ



ในแวดวงคอมพิวเตอร์ Algorithm ถือเป็นหัวใจสำคัญของการเขียนโปรแกรม เนื่องจากคอมพิวเตอร์ทำงานโดยการประมวลผลตามคำสั่งอย่างเป็นลำดับขั้นตอน ดังนั้นนักพัฒนาจึงต้องออกแบบ Algorithm ที่ดีเพื่อแก้ปัญหาต่างๆ ก่อนจะนำมาเขียนเป็นโค้ดคอมพิวเตอร์

4.1 ประโยชน์ของอัลกอริทึม (Algorithm)

1. **ลดความผิดพลาด:** เมื่อมีขั้นตอนที่ชัดเจน เราจะทำงานไม่ข้ามขั้นตอน
2. **ทำงานแทนกันได้:** เพื่อนหรือหุ่นยนต์สามารถมาทำต่อจากเราได้ทันทีเพียงแค่อ่าน Algorithm ของเรา
3. **ตรวจสอบง่าย:** ถ้างานพัง เราจะรู้ทันทีว่าพังที่ขั้นตอนไหน (เรียกว่าการ Debug)
4. **ทำซ้ำได้:** ไม่ว่าจะทำกี่ครั้ง ผลลัพธ์ก็จะออกมาดีเหมือนเดิมทุกครั้ง

4.2 ความสำคัญของอัลกอริทึม (Algorithm)

การเขียนโปรแกรม: Algorithm เป็นหัวใจสำคัญของการเขียนโปรแกรม ช่วยให้นักพัฒนาสามารถออกแบบ เขียน ทดสอบ และบำรุงรักษาโปรแกรมได้อย่างมีประสิทธิภาพ การ

วิเคราะห์ข้อมูล: Algorithm ช่วยให้นักวิเคราะห์สามารถรวบรวม จัดระเบียบ วิเคราะห์ และตีความข้อมูล นำเสนอข้อมูลได้อย่างมีประสิทธิภาพ

การแก้ปัญหา: Algorithm ช่วยให้เราสามารถระบุ วิเคราะห์ หาสาเหตุ หาวิธีแก้ไข และประเมินผลลัพธ์ของปัญหาได้อย่างมีประสิทธิภาพ

การตัดสินใจ: Algorithm ช่วยให้เราสามารถรวบรวม วิเคราะห์ ประเมินตัวเลือก เลือกทางเลือกที่ดีที่สุด และรับผิดชอบต่อผลลัพธ์ของการตัดสินใจได้อย่างมีประสิทธิภาพ

การคิดสร้างสรรค์: Algorithm ช่วยให้เราสามารถคิดวิเคราะห์ คิดอย่างมีวิจารณญาณ หาทางออกใหม่ ๆ คิดริเริ่ม และสร้างสิ่งใหม่ ๆ ได้อย่างมีประสิทธิภาพ

4.3 การพัฒนาทักษะ Algorithm

การพัฒนาทักษะ Algorithm เป็นสิ่งที่ทุกคนควรให้ความสำคัญ ทักษะเหล่านี้ ช่วยให้เราสามารถแก้ปัญหา วิเคราะห์ข้อมูล ตัดสินใจ คิดสร้างสรรค์ และทำงานได้อย่างมีประสิทธิภาพ มีหลายวิธีที่ช่วยให้เราพัฒนาทักษะ Algorithm

1. การเรียนรู้หลักการของ Algorithm ศึกษา ทำความเข้าใจ หลักการ 4 ประการของ Algorithm ประกอบไปด้วย การย่อยปัญหา การจดจำรูปแบบ การออกแบบอัลกอริทึม และการสรุปผล ฝึกฝนหลักการของ Algorithm ผ่านโจทย์ หรือ กิจกรรมต่างๆ
2. การฝึกฝนการแก้ปัญหา โดยใช้ Algorithm หาโจทย์ หรือ ปัญหาต่างๆ วิเคราะห์ปัญหา ออกแบบ Algorithm ทดสอบ Algorithm และแก้ไข Algorithm
3. การเรียนรู้การเขียนโปรแกรม ช่วยให้เข้าใจ และ ประยุกต์ใช้ Algorithm เรียนรู้ภาษาโปรแกรม ฝึกฝน เขียนโปรแกรม และ พัฒนา โปรแกรม
4. การเข้าร่วมกิจกรรม โครงการ หรือ การแข่งขัน ที่เกี่ยวข้องกับ Algorithm หา กิจกรรม โครงการ หรือ การแข่งขัน เตรียมตัว เข้าร่วม และ เรียนรู้
5. การหาแหล่งข้อมูล หา แหล่งข้อมูล เว็บไซต์ หนังสือ วิดีโอ และ คอร์สออนไลน์ ศึกษา และ ฝึกฝน

School Bot

ใบงานที่ 4 : วิศวกรวางแผน The Algorithm Master (10 นาที | 10 คะแนน)

ชื่อ: _____เลขที่_____ชั้น_____

คำชี้แจง: ให้นักเรียนออกแบบลำดับขั้นตอน (Algorithm) เพื่อแก้สถานการณ์ที่ได้รับ โดยเขียนเป็นข้อๆ หรือวาดเป็นลูกศร

ภารกิจที่ 1: ช่วยชีวิตตัวเองในเกม! (Game Logic)

นักเรียนต้องเขียนขั้นตอนให้ตัวละครในเกม "เดินไปหยิบกุญแจ แล้วย้อนกลับมาเปิดประตู" โดยใช้คำสั่ง: [เดินหน้า], [หันซ้าย], [หันขวา], [หยิบ], [ไขกุญแจ]

1. _____
2. _____
3. _____
4. _____
5. _____

ภารกิจที่ 2: โปรแกรมแขนกล xArm (Robot Sequence)

จงเรียงลำดับขั้นตอนการ "ย้ายกล่อง" ต่อไปนี้ให้ถูกต้อง (ใส่หมายเลข 1-5 หน้าข้อ)

[ ] (_4_) หมุนหุ่นยนต์ไปยังตำแหน่งที่ต้องการวาง

[ ] (_2_) สั่งให้มือคีบ (Gripper) ปิดเพื่อคีบกล่อง

[ ] (_1_) เคลื่อนที่แขนกลลงไปเหนือกล่อง

[ ] (_5_) สั่งให้มือคีบเปิดเพื่อวางกล่อง

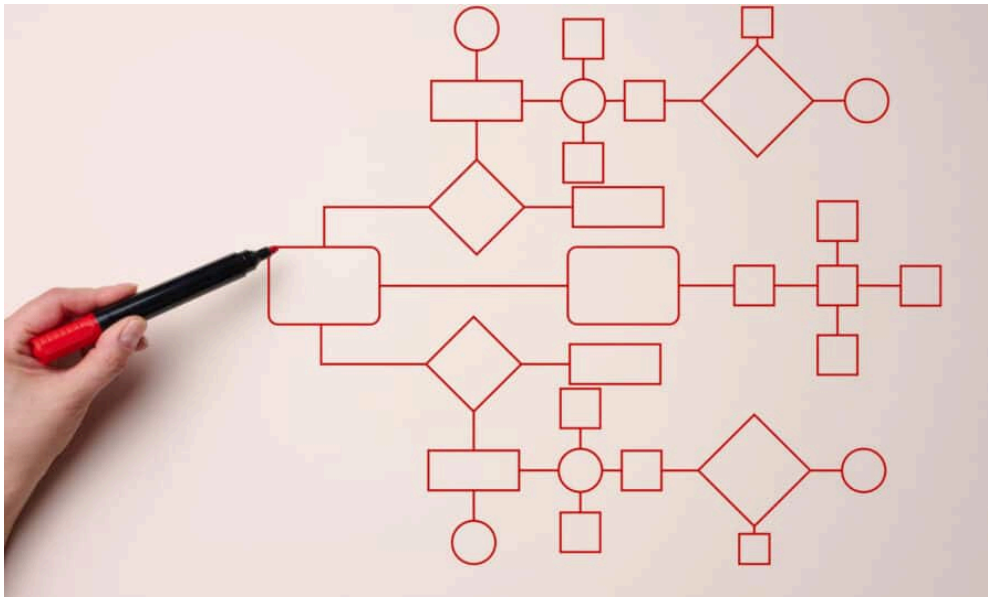
[ ] (_3_) ยกแขนกลขึ้นเพื่อไม่ให้กล่องลากไปกับพื้น

5. Flow chart

นำเข้าสู่บทเรียน (10 นาที)

"นักเรียนครับ คาบที่แล้วเราเขียนขั้นตอนการทำงานเป็นข้อๆ ไปแล้ว (Algorithm) แต่ถ้าขั้นตอนมันยาวเป็น 100 ข้อล่ะ? เราคงอ่านกันตาลายแน่ๆ"

"วันนี้ครูมี 'ภาษาภาพ' มาฝากครับ มันคือการใช้กล่องรูปทรงต่างๆ มาเชื่อมต่อกันด้วยลูกศร เพื่อบอกหุ่นยนต์ว่า 'ถ้าเจอทางแยก ให้เลี้ยวขวา' หรือ 'ถ้าของยังไม่เต็ม ให้กลับไปหยิบใหม่' สิ่งนี้เราเรียกว่า Flowchart ครับ"



Flow chart

Flow chart หรือ ผังงาน คือ แผนภูมิที่แสดงลำดับขั้นตอนของกระบวนการ logic หรืออัลกอริทึม โดยใช้สัญลักษณ์ต่างๆ เชื่อมต่อกัน ช่วยให้เข้าใจขั้นตอน วิเคราะห์ปัญหา และแก้ปัญหาได้ง่ายขึ้น Flow chart ประกอบด้วยสัญลักษณ์มาตรฐานที่ใช้แทนขั้นตอนต่างๆ เช่น การเริ่มต้น/สิ้นสุดกระบวนการ, กระบวนการ, การตัดสินใจ, การป้อนข้อมูล/แสดงผล เป็นต้น ซึ่งจะเชื่อมโยงกันด้วยลูกศรแสดงทิศทางการไหลของข้อมูล

5.1 ประเภทของ Flowchart

Flowchart มีหลายประเภท แต่ละประเภทมีวัตถุประสงค์และรูปแบบการใช้งานแตกต่างกัน ประเภทที่พบบ่อย ได้แก่

- System flowchart: แสดงภาพรวมของระบบทั้งหมด
- Program flowchart: แสดงลำดับขั้นตอนของโปรแกรม
- Data flowchart: แสดงการไหลของข้อมูล
- Document flowchart: แสดงการไหลของเอกสาร

5.2 สัญลักษณ์ที่ใช้ใน Flowchart

Flowchart ประกอบด้วยสัญลักษณ์ต่างๆ เชื่อมต่อกัน ในบทความนี้จะแสดง 5 สัญลักษณ์พื้นฐานที่ใช้ทั่วไป ดังนี้

1. Terminator: สัญลักษณ์เริ่มต้น และสิ้นสุด

สัญลักษณ์เริ่มต้น และสิ้นสุด ลักษะเป็นสี่เหลี่ยมผืนผ้าขอบมน หรือขบโค้ง ใช้กำหนดจุดเริ่มต้น และจุดสิ้นสุดของกระบวนการทำงาน



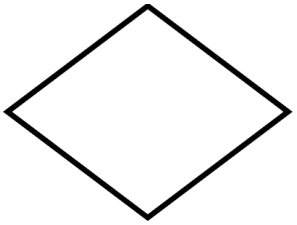
2. Process: สัญลักษณ์แสดงกระบวนการทำงาน

สัญลักษณ์แสดงกระบวนการทำงาน เป็นรูปสี่เหลี่ยมผืนผ้า ใช้เขียนแสดงขั้นตอนหรือกระบวนการทำงานโดยใช้ 1 สัญลักษณ์ต่อ 1 กระบวนการ เช่น ต้มน้ำ, เทน้ำใส่แก้ว ฯลฯ



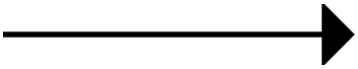
3. Decision: สัญลักษณ์แสดงเกณฑ์ของทางเลือกเพื่อการตัดสินใจ

สัญลักษณ์แสดงการตัดสินใจ ลักษณะเป็นรูปสี่เหลี่ยมข้าวหลามตัด ใช้งานโดยการเขียนเงื่อนไขการเปรียบเทียบ หรือเกณฑ์ในการเลือก



4. Connector: ลูกศรเชื่อมต่อขั้นตอนขั้นตอน

ลูกศรเชื่อมต่อขั้นตอน เป็นลูกศรสำหรับชี้บอกลำดับขั้นตอนในการทำงานจากขั้นตอนหนึ่ง ไปยังอีกขั้นตอนหนึ่ง โดยลากเส้นจากขั้นตอนก่อนหน้า และหัวลูกศรจะชี้อยู่ติดกับสัญลักษณ์ของขั้นตอนถัดไปเสมอ

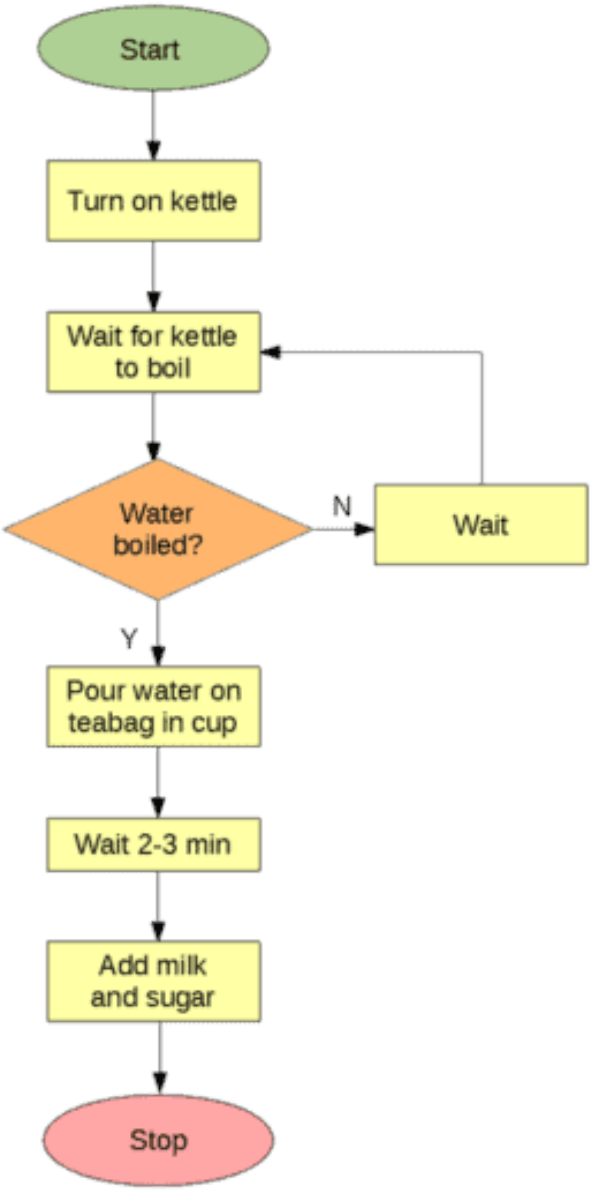


5. Input/Output: สัญลักษณ์แสดงข้อมูลเข้า/ออก

สัญลักษณ์แสดงข้อมูลเข้า/ออก ลักษณะเป็นรูปสี่เหลี่ยมคางหมู ใช้แสดงขั้ยตอนกระบวนการรับ หรือส่งข้อมูลออก เช่น ใช้กับกระบวนการที่คอมพิวเตอร์ต้องรับข้อมูลจากผู้ใช้งาน หรือให้คอมพิวเตอร์แสดงข้อมูลให้ผู้ใช้งานทราบ



ตัวอย่างขั้นตอนการชงชานม



5.3 ประโยชน์ของ Flowchart

1. **เห็นภาพรวม:** ช่วยให้มองออกว่าโปรแกรมมีลำดับอย่างไรจากต้นจนจบ
2. **ตรวจหาจุดผิดง่าย (Easy Debugging):** ถ้าหุ่นยนต์ทำงานผิด เราแค่ไล่ตามลูกศรไปก็จะเจอว่า "กล่องไหน" ที่เป็นปัญหา
3. **สื่อสารเข้าใจง่าย:** ไม่ว่าใครมาอ่าน (เพื่อนหรือครู) ก็จะเข้าใจลำดับความคิดของเราทันทีโดยไม่ต้องอธิบายปากเปล่า
4. **ช่วยในการตัดสินใจ:** เห็นเงื่อนไข (ทางแยก) ได้ชัดเจนกว่าการเขียนเป็นตัวหนังสือ

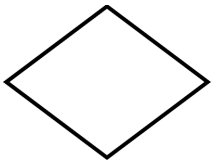
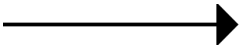
School Bot

ใบงานที่ 5 เรื่อง สัญลักษณ์ Flow chart (5 คะแนน)

ชื่อ: _____**เลขที่**_____**ชั้น**_____

ตอนที่ 1 ความเข้าใจความหมายสัญลัญณ์

คำสั่ง: จับคู่ “รูปทรง” กับ “ความหมาย”

รูปทรง	**ความหมาย**
	• เริ่มต้น/จบ
	• การตัดสินใจ
	• ทิศทาง
	• การทำงาน
	• สัญลักษณ์แสดงข้อมูลเข้า/ออก

School Bot

ใบงานที่ 6 เรื่อง การวางผังงานFlow chart (10 คะแนน)

ชื่อ: _____**เลขที่**_____**ชั้น**_____

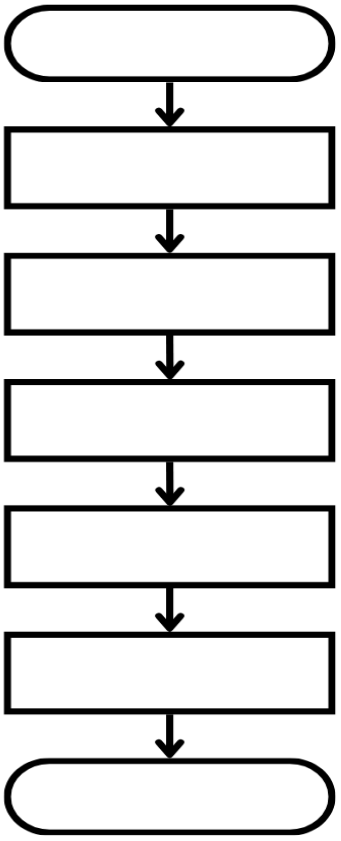
คำชี้แจง จงแสดงขั้นตอนการจำ ลองความคิดตามเงื่อนไขที่โจทย์กำหนด

ตอนที่ 2 ฝึกทักษะการอ่านผังงาน (Reading skill)

1. จงพิจารณาข้อความแสดงขั้นตอนการจำลองความคิดข้างล่างด้านซ้ายมือแล้วนำมาเขียนให้เรียงลำดับเป็นขั้นตอนที่ถูกต้องในช่องว่างด้านขวามือ

เริ่มต้น จบ รับประทาน ต้มน้ำ ให้เดือด ต้มประมาณ 1 นาที ใส่เครื่องปรุงแล้วยกภาชนะลงจากเตา ใส่บะหมี่ลงในน้ำเดือด	

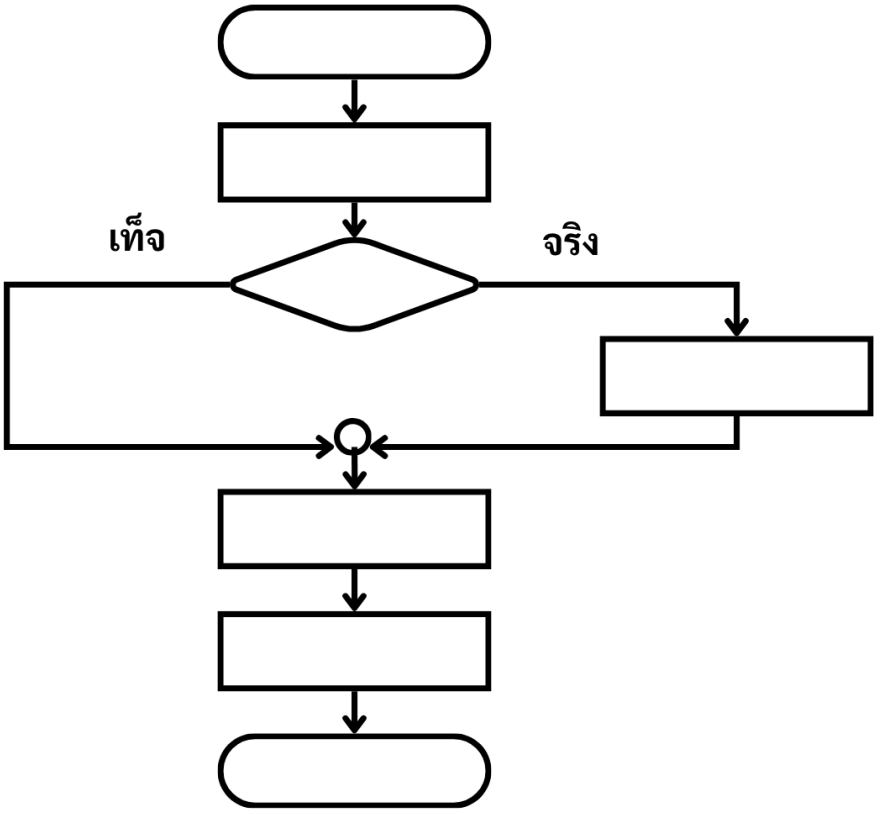
จากขั้นตอนการแก้ปัญหาข้างต้นจงนำข้อความด้านบนมาเติมลงในสัญลักษณ์แต่ละรูป



2. จงพิจารณาข้อความแสดงขั้นตอนการจำลองความคิดข้างล่างด้านซ้ายมือแล้วนำมาเขียนให้เรียงลำดับเป็นขั้นตอนที่ถูกต้องในช่องว่างด้านขวามือ

เริ่มต้น จบ ง่วงใช่ไหม อาบน้ำ แต่งตัว ถ้าง่วง นอนต่ออีก10 นาที ตื่นนอน ไปโรงเรียน	

จากขั้นตอนการแก้ปัญหาข้างต้นจงนำข้อความด้านบนมาเติมลงในสัญลักษณ์แต่ละรูป

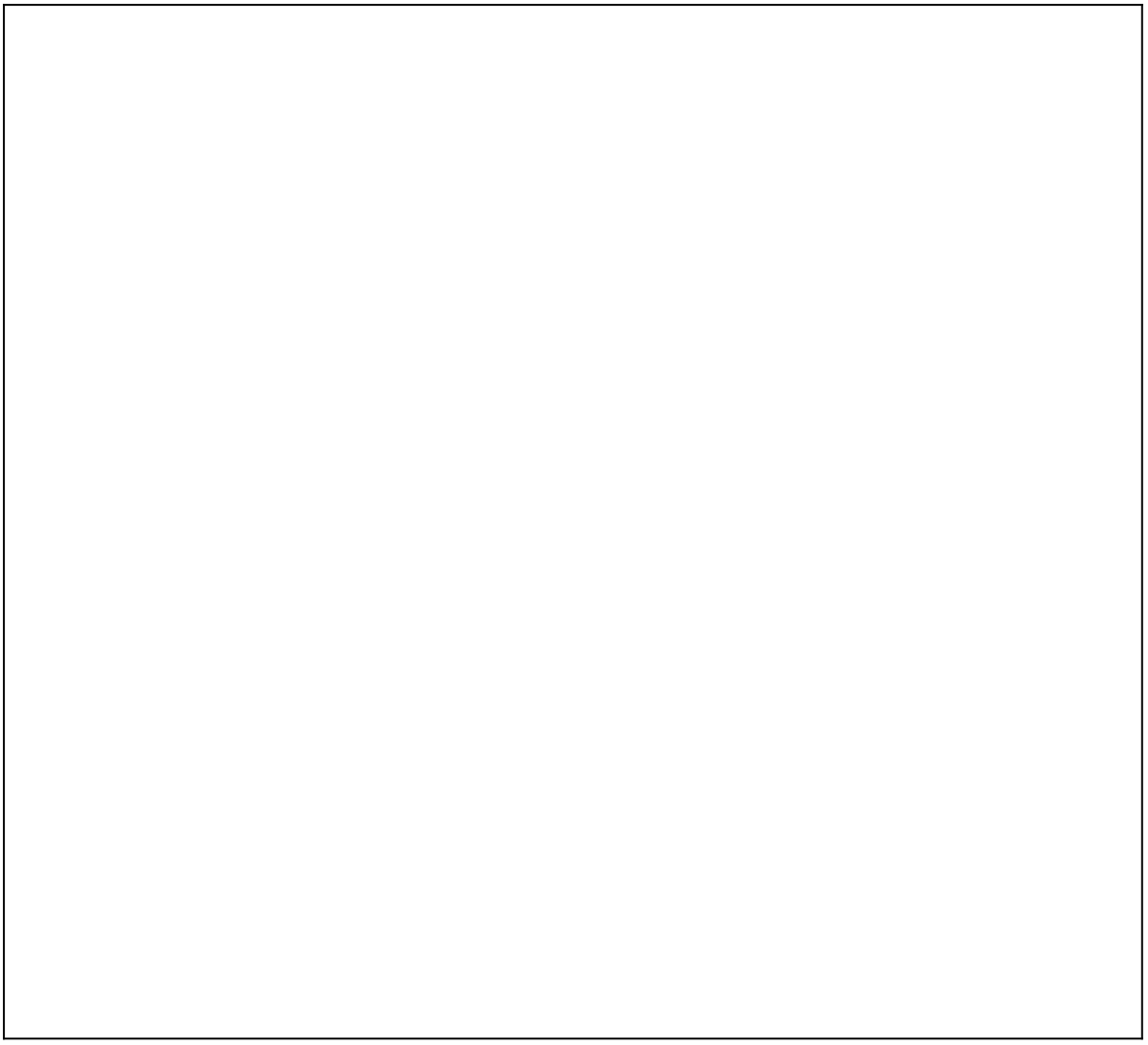


ใบงานที่ 7 เรื่อง การเขียนผังงานFlow chart (20 คะแนน)

ชื่อ: _____เลขที่_____ชั้น_____

ตอนที่ 3 ฝึกทักษะการสร้าง (Construction skill)

3. ให้เรียงลำดับ “การเตรียมตัวไปโรงเรียน” ให้ถูกต้อง และวาดเป็น Flowchart
    - ตื่นนอน
    - อาบน้ำ
    - แต่งตัว
    - กินข้าว
    - แปรงฟัน
    - ใส่รองเท้า
    - ออกจากบ้าน
    - ไปโรงเรียน



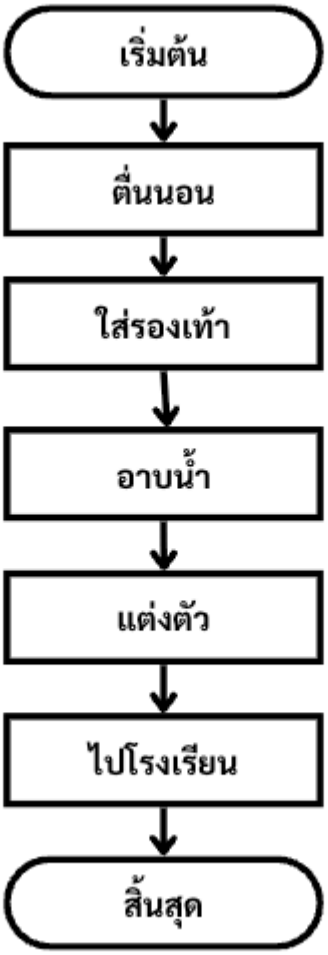
4. ให้นักเรียนวาด Flowchart ของเหตุการณ์ด้านล่าง โดยจะต้องมี Dicision 1 จุด เชื่อมโยงให้ถูกต้อง และต้องมีอย่างน้อย 8 ขั้นตอน
   เหตุการณ์ : “การออกจากบ้านไปโรงเรียนในวันที่ฝนตก”
   เงื่อนไข:
   - ถ้าฝนตก → กางร่ม
   - ถ้าไม่ตก → ไม่ต้องกาง

ใบงานที่ 8 เรื่อง การหา Debug Flow chart (10 คะแนน)

ชื่อ: _____เลขที่_____ชั้น_____

ตอนที่ 4 ฝึกการหาจุดบกพร่อง (Debugging)

5. ในผังงานด้านล่างนี้ มีจุดที่ "ผิด" อยู่ 1 แห่ง นักเรียนหาเจอหรือไม่ว่าคือตรงไหน?

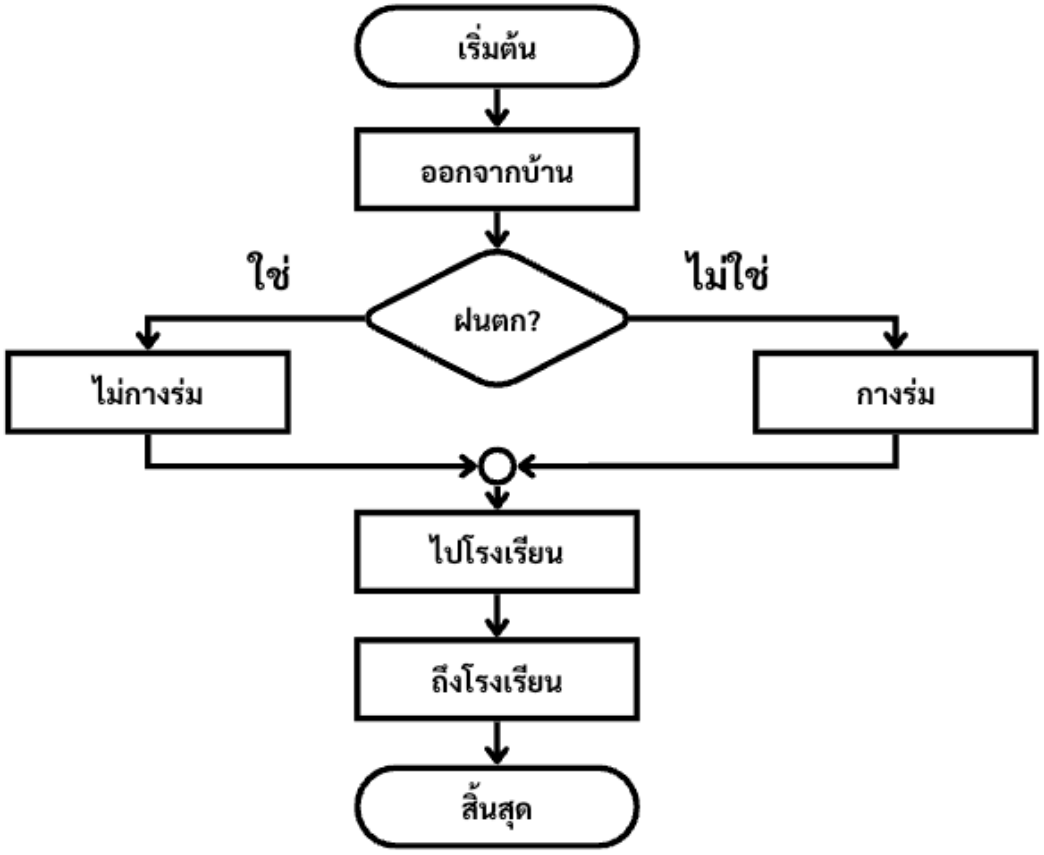


คำถาม

จุดที่ผิดคือจุดไหน: ___ใส่รองเท้า ก่อน อาบน้ำ____เพราะอะไร: ____ไม่สมเหตุสมผล_____

ควรแก้ไขอย่างไรให้ถูกต้อง: _____ควรอยู่หลังแต่งตัว_____

6. ในผังงานด้านล่างนี้ มีจุดที่ "ผิด" อยู่ 1 แห่ง นักเรียนหาเจอหรือไม่ว่าคือตรงไหน?



คำถาม

จุดที่ผิดคือจุดไหน: ____Decision ผิดทาง____เพราะอะไร: _____(Logic กลับด้าน)_____

ควรแก้ไขอย่างไรให้ถูกต้อง: _____สลับ Logic(ไม่ใช่ - ใช่)_____

แบบทดสอบ (Pre-Test/Post-test)
หน่วยการเรียนรู้: แนวคิดเชิงคำนวณ (Computational Thinking)
ระดับชั้น: ประถมศึกษาตอนปลาย

คำชี้แจง: ให้นักเรียนเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว

1. "กระบวนการคิดเพื่อแก้ปัญหาอย่างเป็นระบบ โดยแบ่งปัญหาที่ซับซ้อนให้กลายเป็นส่วนย่อยๆ" คือนิยามของข้อใด?
    ก. การเขียนโปรแกรมคอมพิวเตอร์
    ข. แนวคิดเชิงคำนวณ (Computational Thinking)
    ค. การซ่อมแซมระบบไฟฟ้า
    ง. การใช้งานหุ่นยนต์ไร้สาย
2. องค์ประกอบใดของแนวคิดเชิงคำนวณที่เน้นการ "ตัดรายละเอียดที่ไม่สำคัญออก" เพื่อให้เหลือเพียงใจความสำคัญ?
    ก. การแบ่งปัญหาใหญ่เป็นปัญหาย่อย (Decomposition)
    ข. การพิจารณารูปแบบ (Pattern Recognition)
    ค. การคิดเชิงนามธรรม (Abstraction)
    ง. การออกแบบอัลกอริทึม (Algorithm)
3. หากนักเรียนต้องการสั่งหุ่นยนต์คีบบล็อกไม้ โดยเริ่มจาก "หันฐาน -> ยืดแขน -> คีบ -> ยกขึ้น" การแบ่งขั้นตอนเช่นนี้เรียกว่าอะไร?
    ก. การแบ่งปัญหาใหญ่เป็นปัญหาย่อย (Decomposition)
    ข. การคิดเชิงนามธรรม (Abstraction)
    ค. การพิจารณารูปแบบ (Pattern Recognition)
    ง. การใช้คำสั่งวนซ้ำ (Loop)
4. เมื่อนักเรียนสังเกตพบว่า "ทุกครั้งที่หุ่นยนต์คีบของเสร็จ จะต้องยกแขนขึ้นก่อนหมุนเสมอ" ทักษะนี้เรียกว่าอะไร?
    ก. การออกแบบอัลกอริทึม (Algorithm)
    ข. การพิจารณารูปแบบ (Pattern Recognition)
    ค. การคิดเชิงนามธรรม (Abstraction)
    ง. การแยกส่วนประกอบ
5. สัญลักษณ์รูป "สี่เหลี่ยมผืนผ้า" [ ] ใน Flowchart มีหน้าที่อะไร?
    ก. เริ่มต้น หรือ สิ้นสุด
    ข. การตัดสินใจ (ใช่/ไม่ใช่)
    ค. การปฏิบัติงานหรือการประมวลผล (Process)
    ง. แสดงจุดเชื่อมต่อ

6. สัญลักษณ์ "สี่เหลี่ยมข้าวหลามตัด"  มักจะใช้ในสถานการณ์ใดใน Flowchart?
    ก. เมื่อต้องการจบการทำงาน
    ข. เมื่อต้องเลือกทางแยกหรือมีการตัดสินใจตามเงื่อนไข
    ค. เมื่อต้องการระบุทิศทางการเดินของหุ่นยนต์
    ง. เมื่อต้องการคำนวณตัวเลข
7. ข้อใดคือประโยชน์ที่ชัดเจนที่สุดของการออกแบบ "Algorithm" ก่อนลงมือเขียนโค้ดจริง?
    ก. ทำให้หุ่นยนต์มีสีสันสวยงามขึ้น
    ข. ช่วยให้การทำงานลำดับขั้นตอนชัดเจนและลดความผิดพลาด
    ค. ช่วยประหยัดแบตเตอรี่ของแท็บเล็ต
    ง. ทำให้หุ่นยนต์เคลื่อนที่ได้ไกลกว่าเดิม
8. ในการวาดแผนที่จากบ้านมาโรงงาน หากนักเรียนไม่วาดรูป "สุนัขข้างทาง" แต่เลือกวาดเฉพาะ "ทางแยกสำคัญ" นักเรียนกำลังใช้ทักษะใด?
    ก. Algorithm
    ข. Decomposition
    ค. Abstraction
    ง. Pattern Recognition
9. "จุดเริ่มต้น (Start)" และ "จุดสิ้นสุด (End)" ในการเขียนผังงาน (Flowchart) จะต้องใช้สัญลักษณ์รูปใด?
    ก. สี่เหลี่ยมจัตุรัส
    ข. วงรี
    ค. วงกลม
    ง. สามเหลี่ยม
10. หากนักเรียนเขียน Flowchart แล้วลืมใส่ "ลูกศร (Flow Line)" จะส่งผลเสียอย่างไร?
    ก. โปรแกรมจะทำงานเร็วเกินไป
    ข. ไม่สามารถทราบทิศทางการทำงานของขั้นตอนต่อไปได้
    ค. หุ่นยนต์จะหยุดทำงานทันที
    ง. ไม่มีข้อใดถูก

เฉลยคำตอบ (Answer Key)

1. ข (นิยามโดยตรงจากแผนการเรียน)
2. ค (Abstraction คือการคัดกรองเฉพาะสาระสำคัญ)
3. ก (เป็นการย่อยงานใหญ่ Pick & Place ให้เป็นขั้นตอนเล็กๆ)
4. ข (เป็นการสังเกตสิ่งที่ซ้ำหรือความสัมพันธ์ของท่าทาง)
5. ค (Process/ปฏิบัติงาน)
6. ข (Decision/การตัดสินใจ)
7. ข (ช่วยให้ทำงานเป็นลำดับและตรวจสอบง่าย)
8. ค (เลือกเฉพาะข้อมูลสำคัญ/จำเป็น)
9. ข (Terminator)
10. ข (ทิศทางหายไป ทำให้สื่อสารลำดับไม่ได้)